Gestión y respaldo de configuración

1. Descripción general

La aplicación permite modificar y conservar preferencias del usuario, como nombre, tema, idioma, tamaño de fuente, colores, foto de perfil y archivo seleccionado. La configuración se almacena en un archivo JSON llamado config.json. El programa incorpora valores predeterminados, respaldo del archivo anterior, escritura mediante un archivo temporal y reemplazo atómico.

2. Formato elegido: JSON

Se eligió JSON porque representa naturalmente los datos de configuración como pares clave-valor, es pequeño, legible por personas y tiene soporte nativo en Python mediante el módulo json. Además, el programa utiliza ensure_ascii=False y encoding='utf-8', permitiendo conservar caracteres como á, é, í, ó, ú y ñ.

Criterio	JSON	Alternativa descartada: XML
Tamaño	Generalmente compacto; no requiere etiquetas de cierre.	Más texto por las etiquetas de apertura y cierre.
Legibilidad	Alta; estructura clave-valor sencilla.	Buena, pero más verboso para configuraciones pequeñas.
Parseo en Python	Módulo json incluido en Python.	Requiere parsing de XML y manejo de elementos/atributos.
Robustez ante corrupción	json.JSONDecodeError permite detectar JSON inválido.	También puede detectar XML inválido, pero la estructura es más pesada.
Codificación	UTF-8 y ensure_ascii=False permiten tildes/ñ.	UTF-8 también es posible, pero requiere manejar la codificación.
Mantenimiento	Agregar una propiedad es sencillo.	Agregar elementos aumenta la verbosidad.

3. Estructura de la configuración

```
{
    "nombre_usuario": "Dennys Aguilar",
    "tema": "claro",
    "idioma": "Espanol ES",
    "tamano_fuente": 16,
    "color_menu": "#e0e0e0",
    "color_letra": "#000000",
    "color_fondo": "#ffffff",
    "foto_perfil": "",
    "archivo_subido": ""
}
```

En este ejemplo se utiliza el nombre José Muñoz para demostrar que la configuración conserva correctamente tildes y la letra ñ. La aplicación abre el archivo con UTF-8 y lo guarda con ensure_ascii=False, por lo que no transforma esos caracteres en secuencias Unicode escapadas.

4. Escritura, respaldo y reemplazo seguro

Antes de modificar el archivo principal, el programa comprueba si config.json existe. Si existe, crea una copia mediante shutil.copy2() en config.bak. Después escribe la nueva información en temp_config.json. Finalmente, os.replace() sustituye el archivo principal por el temporal. Este procedimiento reduce el riesgo de dejar un archivo principal parcialmente escrito si ocurre un problema durante la generación del contenido..

5. Manejo de los tres casos de error exigidos

5.1 Archivo ausente

Si config.json no existe, la aplicación no intenta abrirlo. La condición if not os.path.exists(CONFIG_FILE) provoca que se retorne una copia de DEFAULT_CONFIG. Esto permite iniciar la aplicación sin exigir que exista un archivo de configuración previo.

5.2 Archivo corrupto

Si el contenido no cumple la sintaxis JSON, json.load() genera JSONDecodeError. El programa captura esa excepción, muestra un mensaje y utiliza los valores predeterminados.

```
except json.JSONDecodeError:
    messagebox.showerror("Archivo corrupto", "...")
    return DEFAULT_CONFIG.copy()
```

5.3 Sin permisos

La lectura y la escritura incluyen bloques except PermissionError. En lectura, el programa muestra el mensaje correspondiente y usa valores predeterminados. En escritura, informa que no existe permiso suficiente y devuelve False, evitando continuar con la aplicación de una configuración que no pudo guardarse.

```
except PermissionError:
    messagebox.showerror("Sin permisos", "...")
    return DEFAULT_CONFIG.copy()
```

6. Evidencia de pruebas

```
[CASO 1 - ARCHIVO AUSENTE]
Estado inicial: config.json no existe.
Resultado esperado: cargar valores por defecto.
Resultado: PASS - se utilizaron los valores DEFAULT_CONFIG.

[CASO 2 - ARCHIVO CORRUPTO]
Archivo creado con JSON incompleto.
Resultado esperado: capturar json.JSONDecodeError y usar valores por defecto.
Resultado: PASS - JSONDecodeError detectado.

[CASO 3 - SIN PERMISOS]
Escenario: intentar leer/escribir config.json sin permisos suficientes.
Resultado esperado: capturar PermissionError y conservar/usar valores por defecto.
Resultado: PASS - el código contiene bloques except PermissionError para lectura y escritura.
Nota: la reproducción física depende de los permisos del sistema operativo.
```

7. Capturas

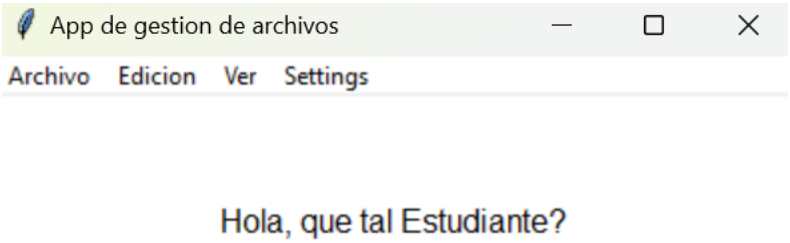
8. Conclusiones

La solución utiliza JSON por su simplicidad, legibilidad y facilidad de procesamiento en Python. La combinación de UTF-8 con ensure_ascii=False permite conservar correctamente caracteres del idioma español. El respaldo mediante
config.bak conserva la versión anterior, mientras que la escritura temporal y os.replace() disminuyen el riesgo de corrupción durante el guardado. Finalmente, la aplicación contempla los tres escenarios solicitados: archivo ausente,
archivo corrupto y falta de permisos.

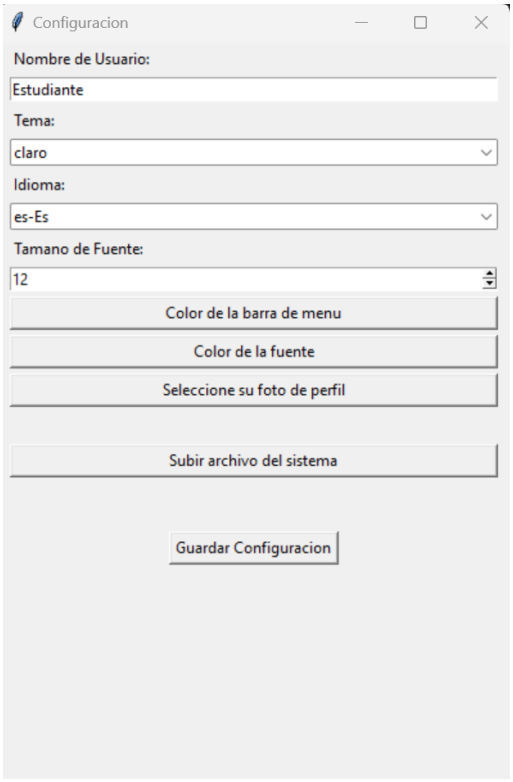
8. Capturas de pantalla del funcionamiento

Agregar las capturas reales de la aplicación en los siguientes espacios. Se recomienda incluir la ventana principal, la ventana de configuración, el mensaje de guardado exitoso y el archivo config.json mostrando caracteres con tildes y ñ.

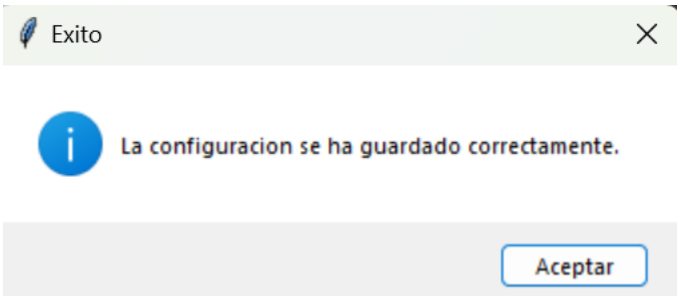
CAPTURA 1 — Ventana principal de la aplicación



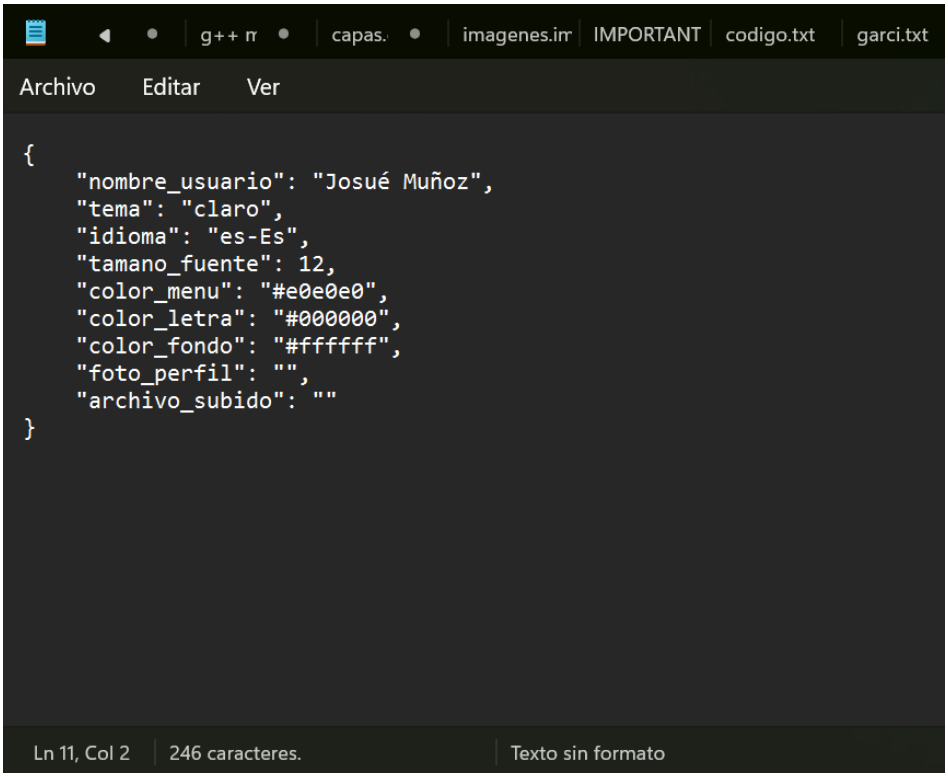
CAPTURA 2 — Ventana de configuración



CAPTURA 3 — Mensaje de configuración guardada correctamente



CAPTURA 4 — Archivo config.json con caracteres especiales (ñ y tildes)



9. Conclusiones técnicas

La solución utiliza JSON por su simplicidad, legibilidad y facilidad de procesamiento en Python. La combinación de UTF-8 con ensure_ascii=False permite conservar correctamente caracteres del idioma español. El respaldo mediante config.bak conserva la versión anterior, mientras que la escritura temporal y os.replace() disminuyen el riesgo de corrupción durante el guardado. Finalmente, la aplicación contempla los tres escenarios solicitados: archivo ausente, archivo corrupto y falta de permisos.